```
import { useState, useEffect } from "react";

const exams = [
  {
    id: 1,
    fach: "Investition & Finanzierung",
    datum: "2026-06-30",
    uhrzeit: "16:00",
    raum: "I001",
    ects: 10,
    farbe: "#e74c3c",
    accent: "#ff6b6b",
    kurz: "I&F",
    wochen: 6,
  },
  {
    id: 2,
    fach: "Informatik 2",
    datum: "2026-07-16",
    uhrzeit: "08:30",
    raum: "I001 & HS 2",
    ects: 10,
    farbe: "#2980b9",
    accent: "#5dade2",
    kurz: "INF",
    wochen: 8,
  },
  {
    id: 3,
    fach: "Physik 1",
    datum: "2026-07-14",
    uhrzeit: "08:30",
    raum: "HS 2",
    ects: 5,
    farbe: "#8e44ad",
    accent: "#bb8fce",
    kurz: "PHY",
    wochen: 8,
  },
  {
    id: 4,
    fach: "Grundlagen Ingenieurmathematik 2",
    datum: "2026-07-21",
    uhrzeit: "08:30",
```

```
    raum: "L106 & HS 2",
    ects: 5,
    farbe: "#27ae60",
    accent: "#58d68d",
    kurz: "MATHE",
    wochen: 9,
  },
];

const today = new Date("2026-05-20");

function getDaysUntil(dateStr) {
  const d = new Date(dateStr);
  return Math.ceil((d - today) / (1000 * 60 * 60 * 24));
}

function getWeeks(dateStr) {
  return Math.floor(getDaysUntil(dateStr) / 7);
}

function generatePlan(exam) {
  const days = getDaysUntil(exam.datum);
  const weeks = Math.floor(days / 7);
  const phases = [];

  if (exam.ects === 10) {
    // Heavy subject
    phases.push({
      phase: "Grundlagen auffrischen",
      dauer: `${Math.round(weeks * 0.3)} Wochen`,
      beschreibung: "Vorlesungsskripte durcharbeiten, Lücken identifizieren",
      tipps: ["Alle Kapitel überfliegen", "Karteikarten für Definitionen anlegen"
    });
    phases.push({
      phase: "Intensivphase",
      dauer: `${Math.round(weeks * 0.4)} Wochen`,
      beschreibung: "Übungsaufgaben lösen, Klausuren aus Vorjahren",
      tipps: ["Täglich mind. 2 Aufgaben rechnen", "Schwache Themen gezielt üben",
    });
    phases.push({
      phase: "Wiederholung & Festigung",
      dauer: `${Math.round(weeks * 0.2)} Wochen`,
      beschreibung: "Alles nochmal durchgehen, Zeitdruck simulieren",
      tipps: ["Alte Klausuren unter Zeitdruck lösen", "Formelblatt finalisieren",
    });
    phases.push({
      phase: "Letzte Woche",
```

```
      dauer: "1 Woche",
      beschreibung: "Ruhig bleiben, Kerninhalte festigen",
      tipps: ["Nur noch Wiederholung, nichts Neues", "Früh schlafen gehen", "Prüf
    });
  } else {
    phases.push({
      phase: "Überblick & Grundlagen",
      dauer: `${Math.round(weeks * 0.35)} Wochen`,
      beschreibung: "Skript komplett durcharbeiten, Zusammenfassung erstellen",
      tipps: ["Eine kompakte Zusammenfassung (max. 10 Seiten)", "Wichtige Formeln
    });
    phases.push({
      phase: "Aufgaben & Vertiefung",
      dauer: `${Math.round(weeks * 0.4)} Wochen`,
      beschreibung: "Übungsaufgaben, Musterlösungen analysieren",
      tipps: ["Aufgabentypen kategorisieren", "Lösungswege auswendig lernen"],
    });
    phases.push({
      phase: "Finish",
      dauer: `${Math.round(weeks * 0.25)} Wochen`,
      beschreibung: "Wiederholung und Probeklausur",
      tipps: ["Probeklausur unter echten Bedingungen", "Formelblatt abschließen"]
    });
  }

  return phases;
}

const weeklyHours = {
  1: { "I&F": 4, INF: 1, PHY: 3, MATHE: 6 },
  2: { "I&F": 5, INF: 2, PHY: 4, MATHE: 7 },
  3: { "I&F": 6, INF: 2, PHY: 5, MATHE: 8 },
  4: { "I&F": 7, INF: 3, PHY: 5, MATHE: 9 },
  5: { "I&F": 9, INF: 3, PHY: 6, MATHE: 10 },
  6: { "I&F": 11, INF: 4, PHY: 7, MATHE: 10 },
};

export default function Lernplan() {
  const [selected, setSelected] = useState(null);
  const [checkedTasks, setCheckedTasks] = useState({});
  const [view, setView] = useState("uebersicht"); // uebersicht | detail | wochen

  const sorted = [...exams].sort((a, b) => new Date(a.datum) - new Date(b.datum))

  const toggleTask = (key) => {
    setCheckedTasks((prev) => ({ ...prev, [key]: !prev[key] }));
  };
```

```
const ExamCard = ({ exam, compact }) => {
  const days = getDaysUntil(exam.datum);
  const weeks = getWeeks(exam.datum);
  const urgency = days < 20 ? "🔴" : days < 40 ? "🟡" : "🟢";

  return (
    <div
      onClick={() => { setSelected(exam); setView("detail"); }}
      style={{
        background: `linear-gradient(135deg, ${exam.farbe}22, ${exam.farbe}11)`
        border: `1px solid ${exam.farbe}44`,
        borderLeft: `4px solid ${exam.farbe}`,
        borderRadius: 12,
        padding: compact ? "12px 16px" : "20px 24px",
        cursor: "pointer",
        transition: "all 0.2s",
        marginBottom: compact ? 8 : 12,
      }}
      onMouseEnter={e => e.currentTarget.style.transform = "translateX(4px)"}
      onMouseLeave={e => e.currentTarget.style.transform = "translateX(0)"}
    >
      <div style={{ display: "flex", justifyContent: "space-between", alignItem
        <div>
          <div style={{ fontWeight: 700, fontSize: compact ? 14 : 16, color: "#
            {urgency} {exam.fach}
          </div>
          <div style={{ fontSize: 13, color: "#555" }}>
            📅 {new Date(exam.datum).toLocaleDateString("de-DE", { weekday: "lc
          </div>
          {!compact && (
            <div style={{ fontSize: 12, color: "#777", marginTop: 2 }}>
              📍 {exam.raum} · {exam.ects} ECTS
            </div>
          )}
        </div>
        <div style={{ textAlign: "right", flexShrink: 0, marginLeft: 12 }}>
          <div style={{
            background: exam.farbe, color: "#fff", borderRadius: 20,
            padding: "4px 10px", fontSize: 13, fontWeight: 700
          }}>
            {days}d
          </div>
          <div style={{ fontSize: 11, color: "#888", marginTop: 3 }}>{weeks} Wo
        </div>
      </div>
    </div>
```

```
  );
};

const DetailView = ({ exam }) => {
  const plan = generatePlan(exam);
  const days = getDaysUntil(exam.datum);

  return (
    <div>
      <button
        onClick={() => { setSelected(null); setView("uebersicht"); }}
        style={{ background: "none", border: "none", color: exam.farbe, cursor:
      >
        ← Zurück zur Übersicht
      </button>

      <div style={{
        background: `linear-gradient(135deg, ${exam.farbe}, ${exam.accent})`,
        borderRadius: 16, padding: "24px", color: "#fff", marginBottom: 24
      }}>
        <div style={{ fontSize: 22, fontWeight: 800, marginBottom: 4 }}>{exam.f
        <div style={{ fontSize: 14, opacity: 0.9 }}>
          {new Date(exam.datum).toLocaleDateString("de-DE", { weekday: "long",
        </div>
        <div style={{ fontSize: 13, opacity: 0.8, marginTop: 2 }}>{exam.raum} ·
        <div style={{
          marginTop: 16, background: "rgba(255,255,255,0.2)", borderRadius: 10,
          display: "flex", gap: 24
        }}>
          <div><div style={{ fontSize: 28, fontWeight: 800 }}>{days}</div><div
          <div><div style={{ fontSize: 28, fontWeight: 800 }}>{getWeeks(exam.da
          <div><div style={{ fontSize: 28, fontWeight: 800 }}>{exam.ects}</div>
        </div>
      </div>

      <div style={{ fontSize: 14, fontWeight: 700, color: "#444", marginBottom:
        Lernphasen
      </div>

      {plan.map((phase, i) => {
        const key = `${exam.id}-phase-${i}`;
        const done = checkedTasks[key];
        return (
          <div key={i} style={{
            background: done ? "#f0fdf4" : "#fafafa",
            border: `1px solid ${done ? "#86efac" : "#e0e0e0"}`,
            borderRadius: 12, padding: "16px", marginBottom: 12,
```

```
  transition: "all 0.2s"
}}>
  <div style={{ display: "flex", justifyContent: "space-between", ali
    <div style={{ display: "flex", alignItems: "center", gap: 10 }}>
      <div style={{
        width: 28, height: 28, borderRadius: "50%",
        background: done ? "#22c55e" : exam.farbe,
        color: "#fff", display: "flex", alignItems: "center", justify
        fontSize: 13, fontWeight: 700, flexShrink: 0
      }}>{done ? "✓" : i + 1}</div>
      <div style={{ fontWeight: 700, fontSize: 15, color: done ? "#15
        {phase.phase}
      </div>
    </div>
    <div style={{
      background: `${exam.farbe}22`, color: exam.farbe,
      borderRadius: 20, padding: "3px 10px", fontSize: 12, fontWeight
    }}>
      {phase.dauer}
    </div>
  </div>
  <div style={{ fontSize: 13, color: "#555", marginBottom: 8, padding
    {phase.beschreibung}
  </div>
  <div style={{ paddingLeft: 38 }}>
    {phase.tipps.map((tipp, j) => {
      const taskKey = `${exam.id}-${i}-${j}`;
      const taskDone = checkedTasks[taskKey];
      return (
        <div
          key={j}
          onClick={(e) => { e.stopPropagation(); toggleTask(taskKey);
          style={{
            display: "flex", alignItems: "center", gap: 8, padding: "
            cursor: "pointer", fontSize: 13, color: taskDone ? "#888"
            textDecoration: taskDone ? "line-through" : "none"
          }}
        >
          <div style={{
            width: 18, height: 18, borderRadius: 4, flexShrink: 0,
            border: `2px solid ${taskDone ? "#22c55e" : exam.farbe}`,
            background: taskDone ? "#22c55e" : "transparent",
            display: "flex", alignItems: "center", justifyContent: "c
            color: "#fff", fontSize: 11
          }}>
            {taskDone ? "✓" : ""}
          </div>
```

```
                    {tipp}
                  </div>
                );
              })}
            </div>
          </div>
        );
      })}
    </div>
  );
};

const Timeline = () => {
  const startDate = new Date("2026-05-18");
  const endDate = new Date("2026-07-22");
  const totalDays = Math.ceil((endDate - startDate) / (1000 * 60 * 60 * 24));

  return (
    <div>
      <div style={{ fontSize: 14, fontWeight: 700, color: "#444", marginBottom:
        Prüfungszeitstrahl
      </div>
      <div style={{ position: "relative", height: 80, marginBottom: 32 }}>
        {/* Timeline bar */}
        <div style={{
          position: "absolute", top: 32, left: 0, right: 0, height: 4,
          background: "#e0e0e0", borderRadius: 2
        }} />
        {/* Today marker */}
        <div style={{
          position: "absolute", top: 24, left: 0, width: 2, height: 20,
          background: "#333", borderRadius: 1
        }}>
          <div style={{ position: "absolute", top: 22, left: -15, fontSize: 10,
        </div>
        {/* Exam markers */}
        {sorted.map((exam) => {
          const days = getDaysUntil(exam.datum);
          const pct = (days / totalDays) * 100;
          return (
            <div
              key={exam.id}
              onClick={() => { setSelected(exam); setView("detail"); }}
              style={{
                position: "absolute", top: 22, left: `${pct}%`,
                transform: "translateX(-50%)",
                cursor: "pointer"
```

```
          }}
        >
          <div style={{
            width: 14, height: 14, borderRadius: "50%",
            background: exam.farbe, border: "3px solid #fff",
            boxShadow: `0 0 0 2px ${exam.farbe}`,
            margin: "0 auto"
          }} />
          <div style={{
            fontSize: 9, fontWeight: 700, color: exam.farbe,
            textAlign: "center", marginTop: 4, whiteSpace: "nowrap"
          }}>
            {exam.kurz}
          </div>
          <div style={{ fontSize: 8, color: "#888", textAlign: "center" }}>
            {new Date(exam.datum).toLocaleDateString("de-DE", { day: "2-dig
          </div>
        </div>
      );
    })}
  </div>

  <div style={{ fontSize: 14, fontWeight: 700, color: "#444", marginBottom:
    Empfohlene Wochenstunden (gesamt)
  </div>
  <div style={{ overflowX: "auto" }}>
    <table style={{ width: "100%", borderCollapse: "collapse", fontSize: 12
      <thead>
        <tr style={{ borderBottom: "2px solid #eee" }}>
          <th style={{ textAlign: "left", padding: "8px 4px", color: "#666"
          {sorted.map(e => (
            <th key={e.id} style={{ textAlign: "center", padding: "8px 4px"
              {e.kurz}
            </th>
          ))}
          <th style={{ textAlign: "center", padding: "8px 4px", color: "#33
        </tr>
      </thead>
      <tbody>
        {[1, 2, 3, 4, 5, 6].map((week) => {
          const weekStart = new Date(today);
          weekStart.setDate(weekStart.getDate() + (week - 1) * 7);
          const label = weekStart.toLocaleDateString("de-DE", { day: "2-dig
          let total = 0;
          return (
            <tr key={week} style={{ borderBottom: "1px solid #f0f0f0" }}>
              <td style={{ padding: "7px 4px", color: "#666", fontSize: 11
```

```
                  {sorted.map(e => {
                    const h = (weeklyHours[week] || {})[e.kurz] || 0;
                    total += h;
                    const examDays = getDaysUntil(e.datum);
                    const weekDaysLeft = examDays - (week - 1) * 7;
                    const passed = weekDaysLeft < 0;
                    return (
                      <td key={e.id} style={{ textAlign: "center", padding: "7p
                        {passed ? (
                          <span style={{ color: "#ccc", fontSize: 10 }}>✓</span
                        ) : (
                          <span style={{
                            background: `${e.farbe}22`, color: e.farbe,
                            borderRadius: 10, padding: "2px 8px", fontWeight: 6
                          }}>
                            {h}h
                          </span>
                        )}
                      </td>
                    );
                  })}
                  <td style={{ textAlign: "center", padding: "7px 4px", fontWei
                </tr>
              );
            })}
          </tbody>
        </table>
      </div>
    </div>
  );
};

return (
  <div style={{
    fontFamily: "'Segoe UI', system-ui, sans-serif",
    maxWidth: 480, margin: "0 auto", padding: "20px 16px",
    background: "#f8f9fa", minHeight: "100vh"
  }}>
    {/* Header */}
    <div style={{ marginBottom: 24 }}>
      <div style={{ fontSize: 11, fontWeight: 700, color: "#888", textTransform
        Sommersemester 2026
      </div>
      <div style={{ fontSize: 26, fontWeight: 800, color: "#1a1a2e" }}>
        Prüfungsplan 📚
      </div>
      <div style={{ fontSize: 13, color: "#777", marginTop: 2 }}>
```

```
          4 Prüfungen · Heute: 20.05.2026 · Noch nicht angefangen
        </div>
      </div>

      {view === "uebersicht" && (
        <>
          <Timeline />
          <div style={{ height: 24 }} />
          <div style={{ fontSize: 14, fontWeight: 700, color: "#444", marginBotto
            Prüfungen — nach Datum
          </div>
          {sorted.map((exam) => <ExamCard key={exam.id} exam={exam} />)}
          <div style={{
            marginTop: 20, padding: 16, background: "#fff3cd",
            border: "1px solid #ffc107", borderRadius: 12, fontSize: 13, color: "
          }}>
            💡 <strong>Tipp:</strong> Mathe 2 hat den meisten Stoff — <strong>heu
          </div>
        </>
      )}

      {view === "detail" && selected && <DetailView exam={selected} />}
    </div>
  );
}
```